

82. Découper la boucle du jeu en fonctions

lire, faire évoluer, dessiner : la boucle du jeu tient en trois appels.

1. Ce qu'il faut comprendre

Un vrai jeu a des centaines de lignes dans sa boucle. Les ranger dans des fonctions aux noms clairs rend **main** **lisible d'un coup d'œil** : `lireTouches(); evoluer(); dessiner();` — la boucle du chapitre 3, écrite en français.

Les fonctions partagent l'**état du jeu** : **energie** est **globale**, elle survit d'une image à l'autre et toutes les fonctions la voient. Chaque fonction a **un seul rôle** : `lireTouches` ne dessine pas, `dessiner` ne change pas l'énergie.

`jauge(ligne, valeur)` est une fonction **avec une boucle**, appelée **dans la boucle** du jeu : elle écrit `valeur` # puis complète avec des espaces jusqu'à 10 — sinon, quand l'énergie baisse, les anciens # resteraient à l'écran.

`dessiner` ne fait rien si l'énergie n'a pas changé depuis le dernier dessin : `affichee` retient la valeur montrée. Sans ce test, la jauge serait réécrite à chaque image — onze écritures pour rien, chacune attendant sa pause (chapitre 3) : du temps que le jeu passerait à attendre au lieu de jouer.

`evoluer` fait baisser l'énergie d'un point tous les 20 tours : un compteur, une remise à zéro — la montre du chapitre 3. Et la garde `energie > 0` évite le 255 du chapitre 1. Tous les chapitres se retrouvent ici.

À toi : ajoute une seconde jauge, pour la magie, qui remonte toute seule.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **790 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      60 o ← la console, à l'allumage — la mise en route de la console
jauge()         71 o ← dessiner() — ta fonction
lireTouches()  13 o ← main() — ta fonction
evoluer()      28 o ← main() — ta fonction
dessiner()     36 o ← main() — ta fonction
main()         27 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase   10 o ← texte() — trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
```

```

EcrireNombre      33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles      85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            288 o ← CopierTuiles – 18 lettres sur 43
"##"              1 o ← texte() – un texte gravé
" "               1 o ← texte() – un texte gravé
"A: RECHARGER"    12 o ← texte() – un texte gravé
Lettres gravées : ACEGHR0123456789:~
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    texte()      × 3    51 o sur place, dans jauge(), main() + AdresseCase (10 o), EcrireTexte (10 o)
    nombre()     × 1    10 o sur place, dans dessiner() + EcrireNombre (33 o)
    bouton()     × 1    5 o sur place, dans lireTouches() + LireManette (37 o)
    image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
    jauge()      × 1    11 o sur place, dans dessiner()
    lireTouches() × 1    3 o sur place, dans main()
    evoluer()    × 1    3 o sur place, dans main()
    dessiner()   × 1    3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : une jauge d'énergie qui se vide toute seule ; A la recharge.
//
// Ce qui est nouveau : découper la boucle du jeu en fonctions.
// Chaque image, un jeu fait toujours les trois mêmes choses, dans cet ordre :
// 1. lireTouches() : ce que le joueur demande
// 2. evoluer() : le monde avance (temps, règles)
// 3. dessiner() : l'écran montre le nouvel état

#include <texte> // écrit un texte à l'écran
#include <bouton> // lit un bouton de la manette
#include <nombre> // écrit un nombre en chiffres

```

2. UNE VARIABLE — ENERGIE

```
uint8_t energie = 10; // L'énergie, de 0 à 10.
```

3. UNE VARIABLE — AFFICHEE

```
uint8_t affichee = 255; // rien n'est encore dessiné (255 : aucune énergie ne vaut ça)
```

4. UNE VARIABLE — MINUTEUR

```
uint8_t minuteur = 0; // Compte les images jusqu'à 20.
```

5. UNE FONCTION — JAUGE()

```

void jauge(uint8_t ligne, uint8_t valeur) { // Dessine « valeur » # sur 10 cases.
    for (uint8_t i = 0; i < 10; i++) { // les 10 cases, colonnes 8 à 17
        if (i < valeur) texte(8 + i, ligne, "#"); // avant la valeur : plein
        else texte(8 + i, ligne, " "); // après : vide
    }
} // jauge(5, 3) → ### (3 pleins, 7 vides)

```

6. UNE FONCTION — LIRETOUCHES()

```

void lireTouches() {
    if (bouton(A)) energie = 10; // on recharge
}

```

7. UNE FONCTION — EVOLUER()

```
void evoluer() {
    minuteur++;                // Une image de plus.
    if (minuteur == 20) {      // Toutes les 20 images (1/3 de seconde) :
        minuteur = 0;         // on recommence à compter,
        if (energie > 0) energie--; // et l'énergie perd 1, sans passer sous 0.
    }
}
```

8. UNE FONCTION — DESSINER()

```
void dessiner() {
    if (energie == affichee) return; // rien n'a changé : on sort tout de suite.
                                     // (return dans une fonction void = « c'est fini ».)
    jauge(5, energie);              // Redessine la jauge sur la ligne 5,
    nombre(1, 5, energie);          // et le nombre à gauche.
    affichee = energie;             // On retient ce qui est à l'écran.
}
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                      // Le jeu commence ici.
    texte(1, 2, "A: RECHARGER");  // La consigne.

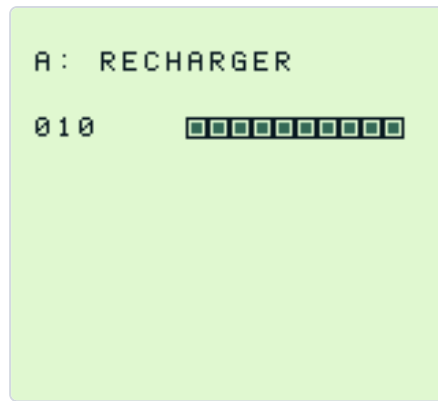
    while (true) {                // La boucle du jeu, maintenant très courte :
        image();                  // attendre l'image,
        lireTouches();            // 1. les touches,
        evoluer();                // 2. le monde,
        dessiner();               // 3. l'écran.
    }
}
```

3. Ce qui se passe, étape par étape

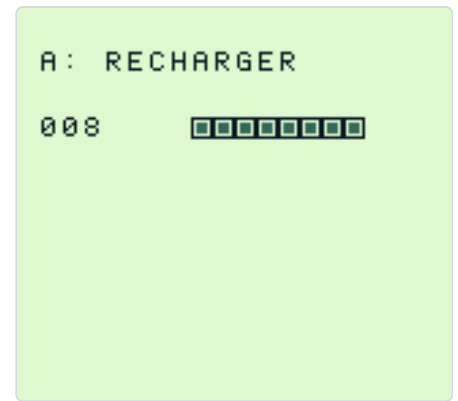
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



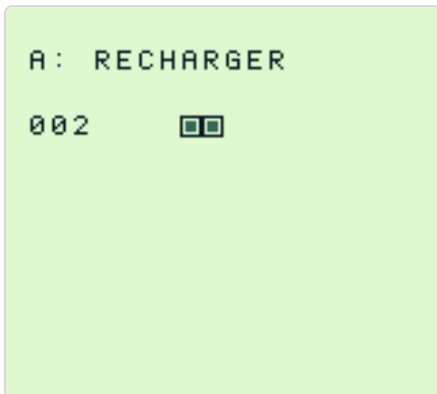
la toute première image — la console vient de démarrer — image 1



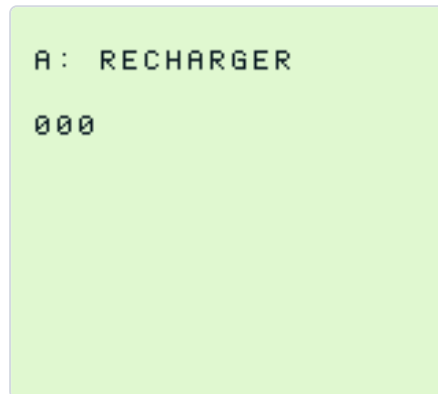
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



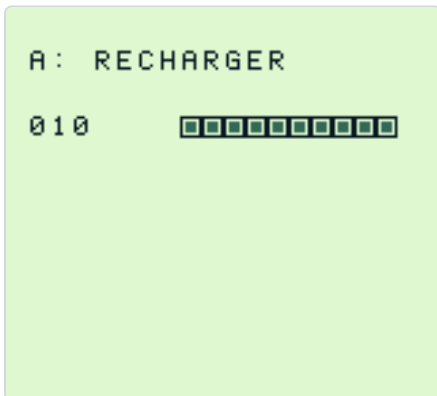
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Une jauge de dix # qui fond toute seule ; A la remplit de nouveau.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'énergie baisse avec le temps (7)
- ✓ la jauge montre autant de # que d'énergie (7 #)
- ✓ elle s'arrête à 0 sans déborder (0)
- ✓ A la recharge (10)

Ensuite : **82.1. Chapitre 13 — Ce qui va ensemble : ce qui vient** — chapitre 13 : Ce qui va ensemble. · gameboy3 — écrire une cartouche Game Boy en C++